

Sistema Multimodal de Recuperación y Búsqueda: Índice Invertido con Codebook frente a las Técnicas Nativas de PostgreSQL

Jyns Arturo Ordóñez Del Carpio, Denzel Bautista, Gian Aedo
Base de Datos II (CS2042) — UTEC — Lima, Perú

Resumen—Presentamos un motor de recuperación por similitud que aplica una única arquitectura — *split* → *extractor* → *codebook* → índice invertido → búsqueda por coseno — a tres modalidades de contenido: texto (18 454 letras de canciones), imagen (44 326 productos de moda) y audio (1 000 pistas de GTZAN). Cada modalidad solo cambia el extractor de características (tokens lingüísticos, SIFT o MFCC) y la construcción del diccionario (top- k de palabras o K-Means, siguiendo el esquema *Bag of Visual Words*); el índice invertido se construye con SPIMI y los histogramas se ponderan con TF-IDF sublineal. Sobre el motor se implementan tres aplicaciones (búsqueda visual e-commerce, búsqueda musical por letra y por similitud acústica, y una consola de consultas multimodal) y se compara contra las técnicas nativas de PostgreSQL: GIN full-text para texto y pgvector con índice HNSW para vectores. Los experimentos muestran que el índice propio y pgvector recuperan exactamente los mismos resultados ($\text{recall} = 1.0$), que el índice propio domina la latencia hasta corpus medianos, y que HNSW acelera la búsqueda vectorial 12–32× con recall de hasta 0.96.

Index Terms—Índice invertido, SPIMI, TF-IDF, Bag of Visual Words, SIFT, MFCC, K-Means, similitud coseno, PostgreSQL, GIN, pgvector, HNSW, búsqueda aproximada.

I. INTRODUCCIÓN

Las bases de datos modernas deben indexar contenido no estructurado — texto, imágenes, audio — y responder consultas por *similitud*, no por igualdad exacta. Este proyecto demuestra que los tres tipos de contenido pueden indexarse y recuperarse con *el mismo paradigma*: dividir el contenido en unidades atómicas (*chunks*), extraer descriptores propios de cada modalidad, cuantizarlos contra un diccionario compartido (*codebook*) y publicar el resultado en un índice invertido consultado por similitud coseno.

Los objetivos son: (1) implementar la arquitectura unificada de punta a punta; (2) persistirla en PostgreSQL; (3) compararla contra los índices nativos del motor (GIN/GiST para texto, pgvector para vectores); y (4) evaluar experimentalmente latencia, throughput, *recall* y memoria para cuantificar los *trade-offs*.

I-A. Aplicaciones implementadas

El backend soporta tres aplicaciones (el enunciado exige al menos dos):

1. **Búsqueda visual e-commerce**: el usuario sube una foto y recibe los productos más similares.

2. **Búsqueda musical**: por letra (full-text) y por similitud acústica (subiendo un audio o eligiendo una pista, con reproducción de las pistas del dataset desde la interfaz).
3. **Consola de consultas multimodal**: un mini-lenguaje tipo SQL que unifica las búsquedas de las tres modalidades.

II. ARQUITECTURA DEL SISTEMA

La Fig. 1 muestra el pipeline común. La Tabla I resume cómo se adapta cada etapa a cada modalidad: el motor de búsqueda (*search_sparse*) es idéntico para las tres.

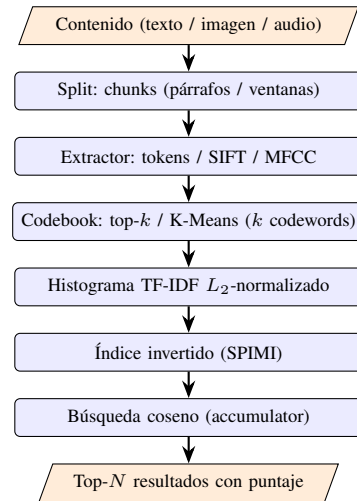


Figura 1. Pipeline unificado: solo el extractor y el codebook dependen de la modalidad; el índice y la búsqueda se comparten.

Cuadro I
ADAPTACIÓN DE CADA ETAPA POR MODALIDAD

Etapa	Texto	Imagen	Audio
Split	estrofas	imagen del producto	ventanas de 3 s
Extractor	tokens (nltk)	SIFT 128-d	MFCC 40-d
Codebook	top- k , $k=1024$	K-Means, $k=512$	K-Means, $k=128$
Ponderación	$\log(1+tf) \cdot idf$, normalización L_2		
Índice	SPIMI → índice invertido (compartido)		
Búsqueda	coseno por acumuladores (compartida)		

III. ESTRUCTURA DEL REPOSITORIO

El código está organizado para que la arquitectura unificada se lea directamente del árbol de carpetas: *el motor* genérico

vive en `engine/` y no sabe nada de canciones ni de productos; las aplicaciones lo orquestan desde `apps/` y lo exponen por HTTP en `api/`.

```
backend/
  app/
    engine/      motor generico (agnostico)
    split/       parrafos / ventanas de audio
    extractor/   sift, mfcc, color hsv
    codebook/    linguistico y k-means
    index/       spimi, indice invertido,
                 histogramas
    search/      busqueda coseno compartida
    query/       parser del mini-lenguaje SQL
  apps/         orquestadores por aplicacion
    music/       text_index, acoustic_index
    ecommerce/   visual_index
  api/          routers FastAPI: music,
                 ecommerce, compare, query
  comparisons/  propio vs GIN vs pgvector
                 (text, image, audio, ann)
  db/           sesion, repositorio y
                 adaptadores de persistencia
  core/         configuracion (k, rutas)
  pipelines/    ingest offline por modalidad
                 (+ --persist a Postgres)
  experiments/  benchmarks Fase 4 y results/
                 (CSV + graficos de este informe)
  tests/        suite pytest (motor y DB)
  frontend/     UI React + Vite (3 pestanas)
  db/init/      01_init.sql: schema + pgvector
                 + indices HNSW y GIN
  docker-compose.yml  db + backend + frontend
  docs/         informe.tex y MANUAL.md
```

La separación tiene una regla simple: `engine/` solo conoce vectores e histogramas (por eso el mismo índice y la misma búsqueda sirven para las tres modalidades); `apps/` conoce los datasets y arma cada índice; `api/` solo traduce HTTP a llamadas del motor; y `db/` es la única capa que habla con PostgreSQL. Los artefactos de índice que produce el *ingest* (codebook, índice invertido y metadatos) se guardan en `data/index/` y los datasets crudos en `data/raw/`; ninguno de los dos se versiona en el repositorio.

IV. ENTREGABLES

- **Repositorio GitHub** con el código fuente documentado y planificación en GitHub Projects.
- **Docker Compose** que levanta PostgreSQL con la extensión `pgvector`, el backend (FastAPI) y el frontend (React), con el esquema inicializado automáticamente desde `db/init/01_init.sql`.
- **Tres aplicaciones funcionando** sobre el mismo motor: búsqueda visual e-commerce, búsqueda musical (letra + similitud acústica con reproducción de pistas) y consola de consultas multimodal.
- **Comparativas de la Fase 3:** GIN full-text y `pgvector` (exacto y HNSW), expuestas como endpoints `/compare/*` para demostrarlas en vivo.
- **Evaluación experimental de la Fase 4:** scripts de benchmark que generan los CSV y los gráficos incluidos en este informe (`backend/experiments/results/`).
- **Documentación:** este informe técnico y un manual de usuario (`docs/MANUAL.md`) con el uso de cada pestaña de la interfaz.

V. INSTALACIÓN Y USO

V-A. Requisitos

Docker (Compose incluido) y una cuenta de Kaggle para descargar los datasets. No hace falta instalar Python ni Node en la máquina: todo corre en contenedores.

V-B. Levantar el sistema

```
docker compose up --build
# db:5432 backend:8000 frontend:5173
```

V-C. Descargar los datasets

Los datasets no están en el repositorio (van en `.gitignore` por tamaño). Se descargan una vez de Kaggle y se dejan en `data/raw/`:

```
# letras de canciones (texto)
kaggle datasets download \
-d imuhammad/audio-features-and-lyrics-of-spotify-songs \
-p data/raw/spotify --unzip

# productos de moda (imagen)
kaggle datasets download \
-d paramaggarwal/fashion-product-images-small \
-p data/raw/fashion --unzip

# pistas GTZAN (audio: csv de features + wavs)
kaggle datasets download \
-d andradaolteanu/gtzan-dataset-music-genre-classification \
-p data/raw/music --unzip
```

V-D. Construir los índices (ingest)

El *ingest* corre una vez por modalidad: construye el codebook y el índice invertido, guarda los artefactos en `data/index/` y, con `--persist`, escribe todo en PostgreSQL:

```
# texto (k=1024)
docker compose exec backend \
python -m pipelines.ingest --app music \
--data /data/raw/spotify/spotify_songs.csv \
--k 1024 --persist

# imagen (k=512 + color)
docker compose exec backend \
python -m pipelines.ingest --app ecommerce \
--modality image \
--data /data/raw/fashion/images \
--k 512 --persist

# audio (k=128)
docker compose exec backend \
python -m pipelines.ingest --app music \
--modality audio \
--data /data/raw/music/features_3_sec.csv \
--k 128 --persist
```

V-E. Usar las aplicaciones

- **Interfaz gráfica:** `http://localhost:5173`, con una pestaña por aplicación.
- **API (Swagger):** `http://localhost:8000/docs`.
- **Comparativas en vivo:**
GET `/compare/text?q=...`,
GET `/compare/vector?external_id=...` y
GET `/compare/audio?filename=...` devuelven los resultados y la latencia de cada técnica para la misma consulta.

V-F. Reproducir los experimentos

```
# comparativa de texto (4 tamanos de corpus)
docker compose exec backend \
  python -m experiments.benchmark \
  --data /data/raw/spotify/spotify_songs.csv \
  --sizes 1000 5000 10000 18454

# HNSW vs busqueda exacta
docker compose exec backend \
  python -m experiments.ann_benchmark
```

Ambos scripts regeneran los CSV y los gráficos de backend/experiments/results/ usados en este informe. La suite de tests se corre con `pytest` desde backend/ (los tests de Postgres se saltan si no hay base disponible).

VI. DATASETS

- **Texto:** *Audio features and lyrics of Spotify songs* (Kaggle): 18 454 canciones con letra, multilingüe.
- **Imagen:** *Fashion Product Images (Small)* (Kaggle): 44 326 fotos de productos de moda (~965 000 descriptores SIFT en total).
- **Audio:** *GTZAN* (Kaggle): 1 000 pistas de 30 s en 10 géneros; se usan las ventanas de 3 s con 20 MFCC (media + varianza = 40 características por ventana) y los .wav para la reproducción desde la interfaz.

El *ingest* acepta .csv, .json y .parquet y mapea columnas por alias, por lo que los datasets son intercambiables.

VII. IMPLEMENTACIÓN POR MÓDULO

VII-A. Extractores y codebook

Texto: tokenización → minúsculas → *stopwords* (ES+EN) → *stemming* (SnowballStemmer); el codebook son las k raíces más frecuentes de la colección. **Imagen:** descriptores locales SIFT y, como complemento, un histograma de color HSV de 32 bins (SIFT trabaja en escala de grises y en moda el color discrimina). **Audio:** MFCC por ventana deslizante. Para imagen y audio, el codebook se obtiene con MiniBatchKMeans sobre todos los descriptores de la colección: los k centroides son las *visual words* / *acoustic words*.

VII-B. Cuantización TF-IDF

Cada chunk se representa como un histograma de codewords ponderado con TF sublineal e IDF, y normalizado en L_2 :

$$w_i = \log(1 + \text{tf}_i) \cdot \log_{10} \frac{N}{\text{df}_i}, \quad (1)$$

donde N es el número de ítems y df_i en cuántos aparece la codeword i . El TF sublineal evita que una codeword muy repetida (el coro de una canción, una textura repetitiva) domine el histograma; el IDF penaliza las poco discriminativas. Con la normalización L_2 , el coseno entre dos histogramas se reduce a un producto punto.

El tamaño k del codebook es la perilla central del *trade-off* calidad/costo: con k chico la cuantización confunde contenidos distintos; con k grande discrimina más, a costa de *posting lists* más largas. Usamos $k=1024$ (texto), $k=512+32$ (imagen

con color) y $k=128$ (audio), respetando el tope de 2 000 dimensiones del HNSW de pgvector.

En imagen, el histograma final concatena la parte visual y la de color manteniendo la norma unitaria: $h = [\sqrt{1-\alpha} \text{BoVW} \parallel \sqrt{\alpha} \text{color}]$, con lo que el coseno queda como promedio ponderado por α entre similitud de forma y de color.

VII-C. Índice invertido con SPIMI

El índice invertido mapea cada codeword a sus *postings* (*chunk*, *peso*). Se construye con SPIMI (Algoritmo 1), que escala a colecciones que no caben en RAM.

Algorithm 1 SPIMI: construcción del índice invertido

- 1: **Entrada:** stream de (*chunk*, *hist disperso*)
 - 2: **Salida:** índice invertido en disco
 - 3: $B \leftarrow \{\}$ {bloque en memoria}
 - 4: **for all** (c, h) en el stream **do**
 - 5: **for all** ($w, peso$) en h **do**
 - 6: agregar ($c, peso$) a $B[w]$
 - 7: **end for**
 - 8: **if** $|B| \geq \text{block_size}$ **then**
 - 9: volcar B a disco *ordenado por w*; $B \leftarrow \{\}$
 - 10: **end if**
 - 11: **end for**
 - 12: volcar el bloque final
 - 13: **return** *k-way merge* de los bloques (heap por w)
-

VII-D. Búsqueda por coseno (accumulator)

La búsqueda (Algoritmo 2) solo visita los chunks que comparten alguna codeword con la consulta, que es lo que hace eficiente al índice invertido frente a un escaneo denso. Es idéntica para las tres modalidades.

Algorithm 2 CosineScore sobre el índice invertido

- 1: **Entrada:** histograma disperso q de la consulta, top- N
 - 2: $S \leftarrow$ mapa de acumuladores en 0
 - 3: **for all** (w, q_w) en q **do**
 - 4: **for all** (c, d_w) en *postings*(w) **do**
 - 5: $S[c] \leftarrow S[c] + q_w \cdot d_w$
 - 6: **end for**
 - 7: **end for**
 - 8: **for all** c en S **do**
 - 9: $S[c] \leftarrow S[c] / (\|q\| \cdot \|d_c\|)$
 - 10: **end for**
 - 11: **return** los N chunks de mayor $S[c]$
-

VII-E. Persistencia en PostgreSQL

El esquema (db/init/01_init.sql) guarda items, chunks, codebook (centroides como vector de pgvector), histograms (histograma por chunk) e inverted_index (postings). Se inicializa automáticamente con Docker Compose.

VIII. COMPARATIVAS NATIVAS EN POSTGRESQL

VIII-A. Texto: GIN full-text

Cada chunk lleva un `tsvector` indexado con GIN; la consulta nativa usa `plainto_tsquery` y rankea con `ts_rank`:

```
SELECT i.id, max(ts_rank(c.tsv, q)) AS rank
FROM chunks c JOIN items i ON i.id=c.item_id,
     plainto_tsquery('spanish', %s) AS q
WHERE c.modality='text' AND c.tsv @@ q
GROUP BY i.id ORDER BY rank DESC LIMIT %s;
```

VIII-B. Vectores: pgvector con HNSW

Los histogramas se consultan por distancia coseno ($\leq$) y se indexan con HNSW, un índice parcial por modalidad cuya dimensión coincide con la k del codebook (texto 1024, imagen 544, audio 128):

```
CREATE INDEX ON histograms
USING hnsw ((hist::vector(1024))
            vector_cosine_ops)
WITH (m=16, ef_construction=64)
WHERE modality='text';
```

Sin el índice, `pgvector` ejecuta un *Seq Scan* (búsqueda exacta); con él, búsqueda aproximada cuya voracidad controla `hnsw.ef_search`.

IX. EVALUACIÓN EXPERIMENTAL

IX-A. Texto: propio vs. GIN vs. pgvector

Se corrió la misma batería de 30 consultas sobre corpus de 1K, 5K, 10K y 18454 chunks (Tabla II); el `recall@10` toma como referencia el ranking coseno del índice propio. Las Figs. 2 y 3 muestran latencia y memoria según el tamaño.

Cuadro II
COMPARATIVA DE TEXTO POR TAMAÑO DE CORPUS ($k=1024$)

Corpus	Método	Media (ms)	p95 (ms)	QPS	R@10
1000	índice invertido	0.44	0.71	2284	1.00
	GIN full-text	1.45	1.97	688	0.07
	pgvector coseno	10.35	12.00	97	1.00
5000	índice invertido	1.21	2.20	824	1.00
	GIN full-text	1.23	1.73	810	0.08
	pgvector coseno	22.99	24.47	44	1.00
10000	índice invertido	2.14	3.64	466	1.00
	GIN full-text	2.51	4.63	399	0.05
	pgvector coseno	25.84	29.91	39	1.00
18454	índice invertido	4.49	7.36	223	1.00
	GIN full-text	3.67	9.54	273	0.04
	pgvector coseno	76.08	88.75	13	1.00

Lectura. El índice propio domina la latencia hasta $\sim 10K$ chunks; en el corpus completo GIN lo empata (3.7 vs. 4.5 ms) porque el vocabulario de $k=1024$ engorda las posting lists propias (889366 postings). `pgvector` recupera *exactamente* lo mismo que el motor propio (`recall = 1.0` en todos los tamaños: ambos rankean por coseno sobre el mismo histograma TF-IDF), pero paga la distancia densa de 1024 dimensiones (76 ms y ~ 400 MB de histogramas frente a ~ 27 MB del GIN). El `recall` de GIN frente al coseno es bajo y cae al crecer el corpus (0.07 $\rightarrow$ 0.04): la coincidencia booleana de tokens responde una pregunta distinta a la del ranking por similitud.

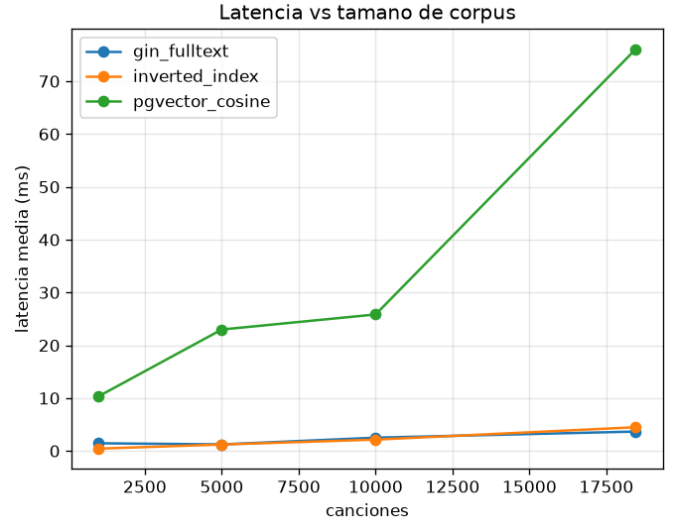


Figura 2. Latencia media según el tamaño del corpus (texto).

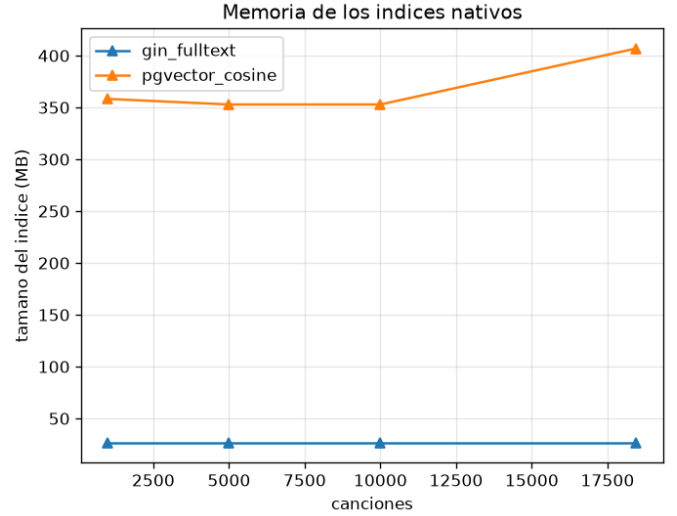


Figura 3. Memoria en disco de los índices nativos.

IX-B. ANN: HNSW vs. búsqueda exacta

Ambas vías ejecutan la misma consulta coseno; solo cambia si el planner usa el índice HNSW o cae a *Seq Scan*. El `recall` del aproximado se mide contra el exacto (Tabla III, Fig. 4).

Cuadro III
HNSW VS. EXACTO (HISTOGRAMAS DE TEXTO, 1024 DIMS)

Método	ef_search	Media (ms)	QPS	R@10
exacto (Seq Scan)	—	47.82	21	1.00
HNSW	10	1.49	670	0.74
HNSW	20	1.48	676	0.85
HNSW	40	2.04	491	0.87
HNSW	80	2.72	368	0.93
HNSW	160	4.02	249	0.96

Lectura. HNSW es 12–32 $\times$ más rápido que el escaneo exacto (47.8 ms $\rightarrow$ 1.5–4 ms) a costa de `recall`, y `ef_search`

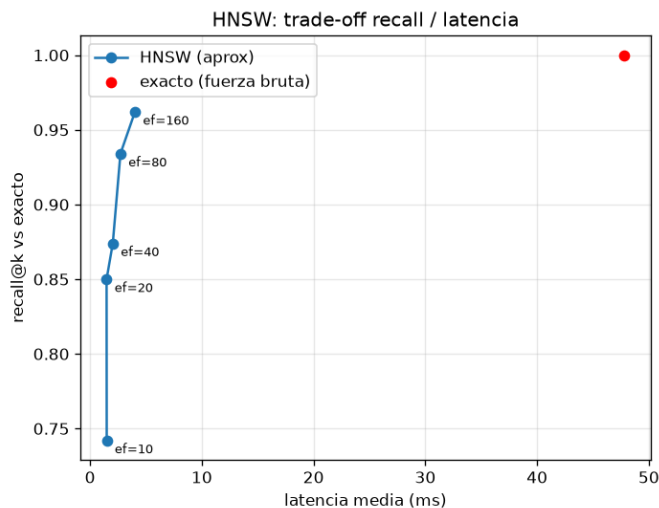


Figura 4. HNSW: curva recall-latencia al variar `ef_search`.

controla el *trade-off*: con 160 se alcanza recall 0.96 a ~ 250 QPS. Con histogramas TF-IDF de 1024 dimensiones el grafo navega mejor que con vocabularios chicos: incluso `ef_search=10` rinde recall 0.74.

IX-C. Audio: propio vs. pgvector

Sobre las 1000 pistas de GTZAN, ambas vías devuelven *exactamente el mismo ranking con los mismos puntajes* (coseno sobre los mismos histogramas de acoustic words). Ejemplo con `blues.00000.wav`: top-3 = {`rock.00071` (0.7321), `blues.00059` (0.6058), `blues.00080` (0.5999)}. La latencia fue ~ 0.4 ms (propio) frente a ~ 10 – 12 ms (pgvector): con histogramas dispersos, el índice invertido compacto responde lo mismo a una fracción del costo.

X. ANÁLISIS DE TRADE-OFFS Y CONCLUSIONES

- **¿Qué técnica ganó en qué métrica?** En latencia, el índice propio gana hasta corpus medianos y GIN lo empata en el corpus completo; en memoria gana el propio (solo postings). En búsqueda vectorial, HNSW domina la latencia (12 – $32\times$) al precio de recall.
- **¿Se recupera la misma información?** El índice propio y pgvector coinciden exactamente (recall = 1.0; en audio con puntajes idénticos) porque comparten la matemática. GIN responde una semántica booleana distinta (recall 0.04–0.08 frente al coseno).
- **Exactitud vs. velocidad.** La cuantización y el ANN introducen falsos positivos/negativos; a cambio el sistema es rápido y compacto. La calidad depende de k y de `ef_search`.
- **Memoria vs. latencia.** pgvector es cómodo (SQL puro) y devuelve lo mismo que el motor propio, pero con $k=1024$ pesa ~ 400 MB y el escaneo denso llega a 76 ms; GIN se mantiene en ~ 27 MB.

Limitaciones. La cuantización pierde detalle fino de los descriptores; las dimensiones de los índices HNSW quedan acopladas a la k del codebook (cambiar una implica reinde-xar); y el histograma de texto degenera si el vocabulario es menor que k .

Recomendación. Usar el índice invertido propio como motor primario (rápido y barato); pgvector+HNSW cuando se quiera delegar la búsqueda vectorial a la base de datos; y reservar GIN para búsqueda literal por palabras clave, no para similitud.